DELHI TECHNOLOGICAL UNIVERSITY

(Formerly Delhi College of Engineering) Shahbad

Daulatpur, Bawana Road, Delhi-110042

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Machine Learning Lab File

CO303

Submitted To:

Assistant Prof. Gull Kaur

Department of Computer Science and Engineering

Submitted By:

Mehul Gupta

24/CS/282

INDEX

SNo.	Aim	Date	Page No.	Signature
1	To implement and evaluate the K-Nearest Neighbors (KNN) classification algorithm on the Iris dataset.			
2	To formulate a machine learning problem, inspect and assess the health of a dataset, separate features and target variables, split the dataset into training and testing sets, and establish a dummy baseline model for evaluating the usefulness of a machine learning model.			
3	To understand and implement proper data preprocessing using Scikit-learn pipelines, identify data leakage in a machine learning workflow, and compare the performance of a leaky preprocessing pipeline with a correctly implemented leakage-free pipeline.			
4	To implement a simple linear regression model, determine the best-fit line between a predictor and target variable, analyze residuals to evaluate linear regression assumptions, and study the effect of outliers on the regression model.			
5	To study polynomial regression and the effect of increasing model complexity on training and testing errors, identify overfitting, and apply Ridge (L2) and Lasso (L1) regularisation to control model complexity and reduce overfitting.			
6				
7				
8				
9				

10				
11				
12				

EXPERIMENT - 5

AIM:

To study polynomial regression and the effect of increasing model complexity on training and testing errors, identify overfitting, and apply **Ridge (L2)** and **Lasso (L1) regularisation** to control model complexity and reduce overfitting.

THEORY:

Polynomial Regression

Polynomial regression extends linear regression by transforming a numerical predictor into polynomial terms such as x, x^2, x^3, and higher powers. The model is still linear in its coefficients, but it can represent curved relationships between the predictor and target.

For degree d, a polynomial model can be written as:

$$y = b_0 + b_1x + b_2x^2 + ... + b_dx^d$$

Model Complexity, Underfitting and Overfitting

Increasing the polynomial degree makes the model more flexible. Training error normally falls as degree increases, but testing error may start to rise if the model begins fitting noise rather than the underlying pattern. A large difference between training and testing error is a **generalisation gap** and is evidence of overfitting.

Ridge and Lasso Regularisation

Ridge regression adds an L2 penalty proportional to the sum of squared coefficients. It shrinks coefficient magnitudes toward zero but usually keeps every term active. **Lasso regression** adds an L1 penalty proportional to the sum of absolute coefficient values and can force some coefficients exactly to zero, producing a sparse model.

The parameter **alpha** controls the strength of regularisation. Very small alpha values give weak regularisation, while very large values can shrink the model too aggressively and cause underfitting.

Dataset and Experimental Setup

The experiment uses the supplied **Housing.csv** dataset with **545 observations and 13 columns**. Following the lab instructions, **area** is used as the numerical predictor and **price** as the target. Both selected columns contain no missing values. The data is divided into **436 training rows** and **109 testing rows** using an 80:20 split with **random_state=42**.

CODE:

```
# EXPERIMENT 5: Polynomial Regression and Regularisation
from pathlib import Path
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import Lasso, LinearRegression, Ridge
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

# Load dataset
data = pd.read_csv("Housing.csv")
X = data[["area"]]
y = data["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
y_train_millions = y_train / 1_000_000

print(f"Training rows: {len(X_train)}")
print(f"Testing rows: {len(X_test)}")
print(f"Area-price correlation: {data['area'].corr(data['price']):.4f}")

def make_polynomial_model(degree, estimator):
    return Pipeline([
        ("polynomial", PolynomialFeatures(degree=degree, include_bias=False)),
        ("scaler", StandardScaler()),
        ("model", estimator),
    ])

def evaluate_model(model):
    train_predictions = model.predict(X_train) * 1_000_000
    test_predictions = model.predict(X_test) * 1_000_000
    return {
        "Training MSE": mean_squared_error(y_train, train_predictions),
        "Testing MSE": mean_squared_error(y_test, test_predictions),
        "Training R2": r2_score(y_train, train_predictions),
        "Testing R2": r2_score(y_test, test_predictions),
    }

# Polynomial regression for several degrees
degrees = [1, 3, 5, 7, 10]
polynomial_models = {}
degree_results = []
for degree in degrees:
    model = make_polynomial_model(degree, LinearRegression())
    model.fit(X_train, y_train_millions)
    polynomial_models[degree] = model
    degree_results.append({"Degree": degree, **evaluate_model(model)})

degree_results_df = pd.DataFrame(degree_results).set_index("Degree")
print(degree_results_df)

# Plot training and testing errors
plt.plot(degrees, degree_results_df["Training MSE"], marker="o", label="Training MSE")
plt.plot(degrees, degree_results_df["Testing MSE"], marker="o", label="Testing MSE")
plt.xlabel("Polynomial degree")
plt.ylabel("MSE")
plt.legend()
plt.show()
```

CODE (CONTINUED):

```
# Ridge and Lasso regularisation of degree 10
alphas = [0.001, 0.01, 0.1, 1, 10, 100]
regularisation_results = []
regularisation_models = {}

for method, factory in {
    "Ridge": lambda a: Ridge(alpha=a),
    "Lasso": lambda a: Lasso(alpha=a, max_iter=200_000, tol=1e-5),
}.items():
    for alpha in alphas:
        model = make_polynomial_model(10, factory(alpha))
        model.fit(X_train, y_train_millions)
        coef = model.named_steps["model"].coef_
        regularisation_models[(method, alpha)] = model
        regularisation_results.append({
            "Method": method,
            "Alpha": alpha,
            **evaluate_model(model),
            "Coefficient L2 Norm": np.linalg.norm(coef),
            "Zero Coefficients": int(np.sum(np.isclose(coef, 0.0, atol=1e-8))),
        })

regularisation_df = pd.DataFrame(regularisation_results)
best_degree_10 = regularisation_df.loc[
    regularisation_df.groupby("Method")["Testing MSE"].idxmin()
]
print(best_degree_10)

# Intermediate degrees with regularisation
intermediate_results = []
for degree in [3, 5, 7]:
    for method, factory in {
        "Ridge": lambda a: Ridge(alpha=a),
        "Lasso": lambda a: Lasso(alpha=a, max_iter=200_000, tol=1e-5),
    }.items():
        for alpha in alphas:
            model = make_polynomial_model(degree, factory(alpha))
            model.fit(X_train, y_train_millions)
            coef = model.named_steps["model"].coef_
            intermediate_results.append({
                "Degree": degree, "Method": method, "Alpha": alpha,
                **evaluate_model(model),
                "Non-zero coefficients": int(np.sum(~np.isclose(coef, 0.0))),
            })

intermediate_df = pd.DataFrame(intermediate_results)
best_intermediate = intermediate_df.loc[
    intermediate_df.groupby(["Degree", "Method"])["Testing MSE"].idxmin()
]
print(best_intermediate.sort_values("Testing MSE"))

# Final selection: lowest testing MSE, then simpler model as tie-breaker
# Result: Degree-3 Lasso, alpha = 0.001
print("Selected model: degree 3 Lasso, alpha=0.001")
```

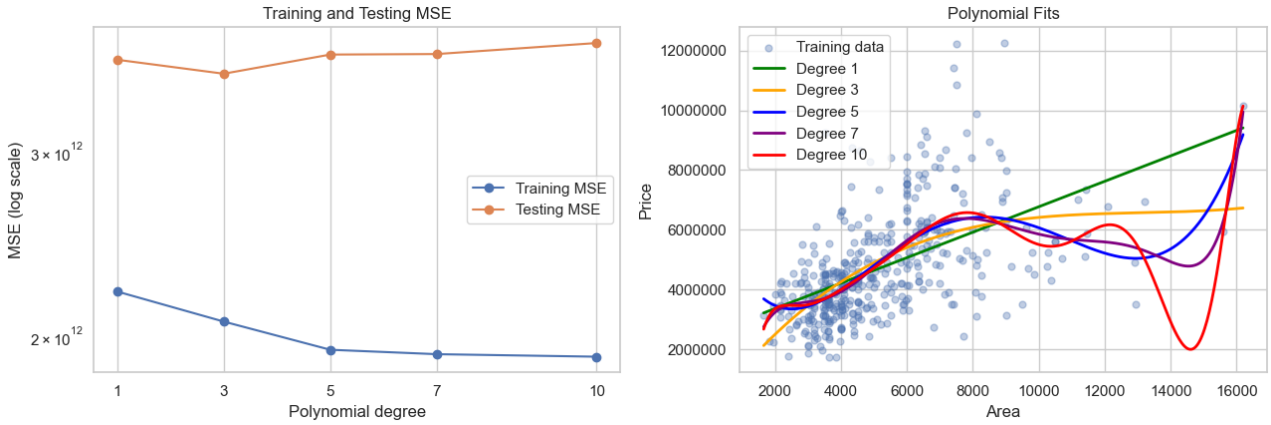
OUTPUT:

The dataset contains **545 rows and 13 columns**. The selected variables are **area** (predictor) and **price** (target), with no missing values in either column. The area-price Pearson correlation is **0.5360**, showing a moderate positive relationship.

Training rows: 436 **Testing rows:** 109 **Split:** 80:20

1. Polynomial Degree Comparison

Degree	Training MSE	Testing MSE	Train R^2	Test R^2
1	2.2047e12	3.6753e12	0.2850	0.2729
3	2.0634e12	3.5638e12	0.3308	0.2949
5	1.9391e12	3.7176e12	0.3711	0.2645
7	1.9203e12	3.7218e12	0.3772	0.2637
10	1.9097e12	3.8133e12	0.3806	0.2456



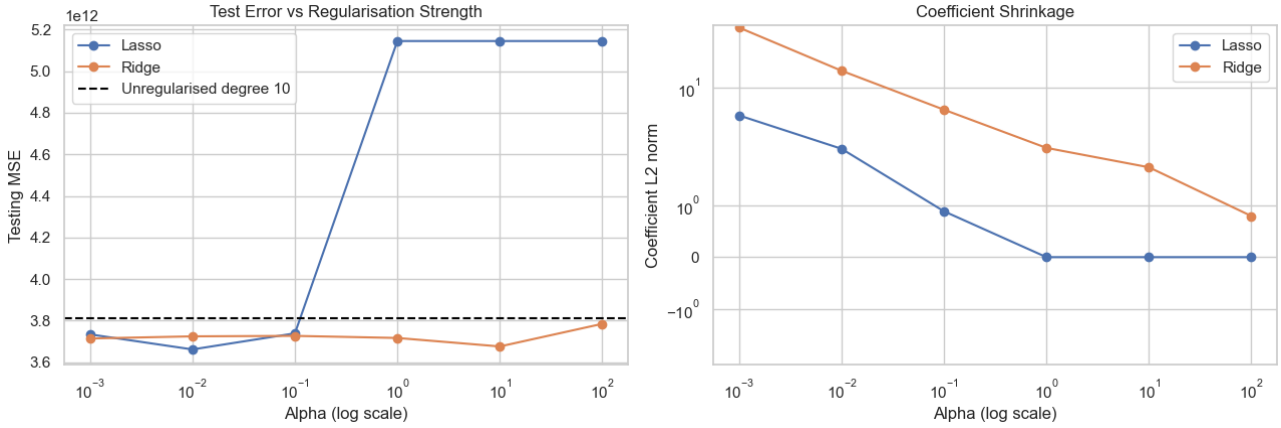
Polynomial models for degrees 1, 3, 5, 7 and 10. Training error keeps decreasing, while testing error is lowest around degree 3 and rises for higher degrees.

The lowest unregularised testing MSE occurs at **degree 3**. At degree 10, the training MSE falls to 1.9097e12 while testing MSE rises to 3.8133e12. The resulting generalisation gap is **1.9035e12**, so degree 10 is treated as the strongest overfit model.

2. Degree-10 Ridge and Lasso Regularisation

Ridge and Lasso were evaluated using alpha values 0.001, 0.01, 0.1, 1, 10 and 100. The best alpha for each method was selected using testing MSE.

Method	Best alpha	Training MSE	Testing MSE	Test R^2	L2 norm	Zero coef.
Lasso	0.01	1.9849e12	3.6616e12	0.2756	2.2223	7 / 10
Ridge	10	1.9904e12	3.6759e12	0.2728	1.7271	0 / 10

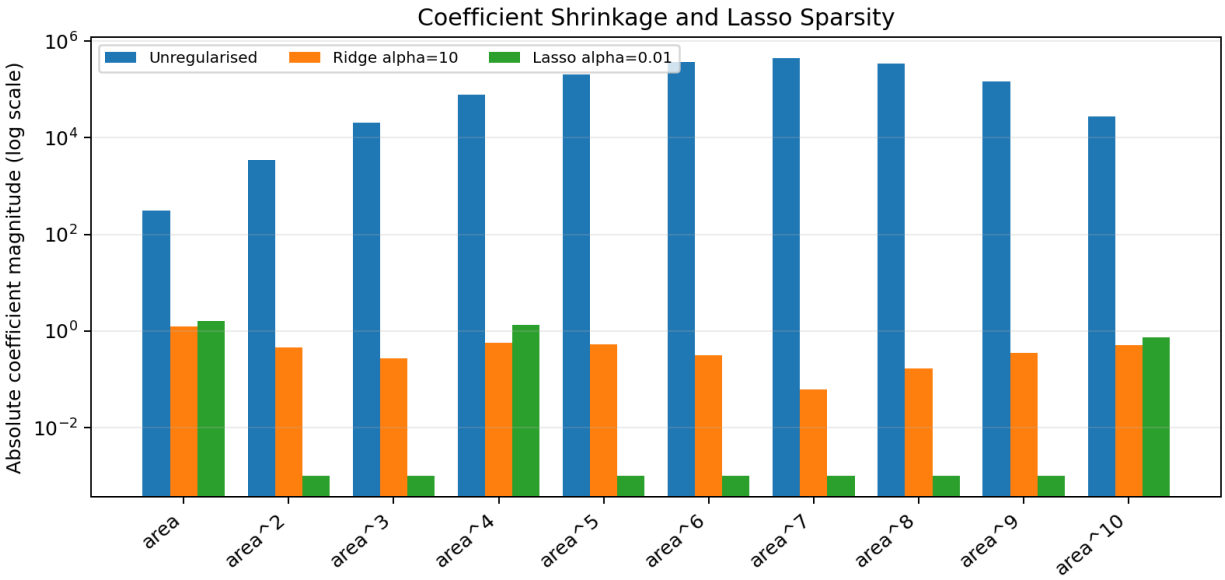


Testing error and coefficient magnitude as the regularisation strength changes.

The unregularised degree-10 coefficient L2 norm was approximately **721,448.59**. Ridge with alpha 10 reduced this to **1.7271**, demonstrating strong coefficient shrinkage. Lasso with alpha 0.01 set **7 of 10** polynomial coefficients exactly to zero. Both regularisers improved the overfit degree-10 model.

3. Coefficient Shrinkage and Sparsity

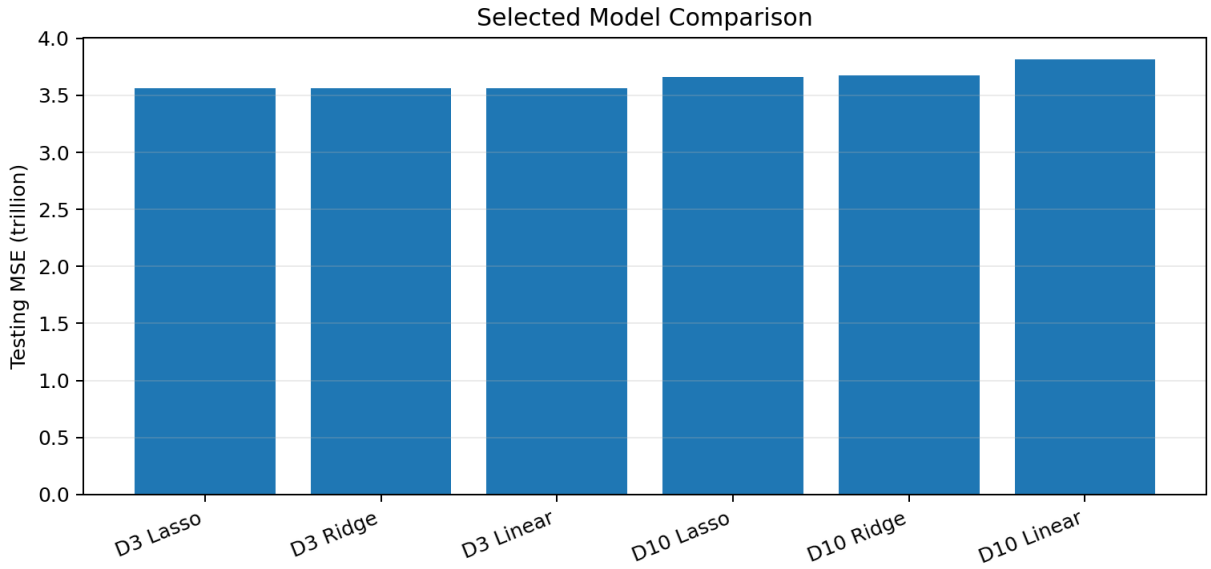
Term	Unregularised	Ridge alpha=10	Lasso alpha=0.01
area	309.7865	1.2516	1.6231
area^2	-3480.0094	0.4527	0.0000
area^3	20358.6439	-0.2707	0.0000
area^4	-78078.2091	-0.5786	-1.3285
area^5	205597.7123	-0.5309	0.0000
area^6	-370238.2357	-0.3147	0.0000
area^7	444573.2418	-0.0620	0.0000
area^8	-338446.7345	0.1679	0.0000
area^9	147161.3658	0.3585	0.0000
area^10	-27755.7443	0.5117	0.7343



The log-scale comparison shows how Ridge compresses all degree-10 terms, while Lasso removes most terms entirely.

4. Intermediate Degrees and Final Model

Degree	Method	Best alpha	Training MSE	Testing MSE	Test R2	Non-zero
3	Lasso	0.001	2.0654e12	3.5622e12	0.2952	3
3	Ridge	0.1	2.0640e12	3.5627e12	0.2952	3
5	Lasso	0.01	2.0445e12	3.5788e12	0.2920	3
5	Ridge	1	2.0129e12	3.5927e12	0.2892	5
7	Ridge	10	2.0232e12	3.6210e12	0.2836	7
7	Lasso	0.01	1.9968e12	3.6228e12	0.2833	3



Testing MSE for the strongest candidate models. Lower values are better.

Selected model: Degree-3 Lasso regression with **alpha = 0.001**. It achieved the lowest observed testing MSE of **3,562,224,275,170.40** and testing R^2 of **0.2952**. Its advantage over unregularised degree 3 is small, but it combines the best held-out performance with low complexity.

OBSERVATIONS:

• Degree 1 has the highest training MSE and may underfit some curvature in the area-price relationship.

• Degree 3 gives the best unregularised test result.

• Degrees 5 and 7 lower training error but increase testing error relative to degree 3, showing that overfitting begins after degree 3.

• Degree 10 has the lowest training MSE and the highest testing MSE among the unregularised models, making it the clearest overfit model.

• Ridge shrinks every coefficient toward zero without producing exact zeros.

• Lasso produces a sparse model by setting polynomial coefficients exactly to zero.

• Increasing alpha increases regularisation strength; very large values cause excessive shrinkage and underfitting.

• Regularisation substantially improves the overfit degree-10 model, but an intermediate degree still generalises better.

• The final model should be chosen primarily using test/validation performance while also considering model complexity.

LEARNING OUTCOME:

• Understand polynomial regression and how polynomial degree changes model complexity.

• Compare training MSE and testing MSE to identify underfitting and overfitting.

• Understand the bias-variance trade-off and the generalisation gap.

• Apply Ridge (L2) and Lasso (L1) regularisation to a high-degree polynomial model.

• Observe coefficient shrinkage with Ridge and sparsity with Lasso.

• Compare regularisation strengths using alpha values and testing performance.

• Use held-out test performance and simplicity to select an appropriate final model.

CONCLUSION:

Increasing polynomial degree reduced training error, but test performance improved only up to degree 3. Higher degrees fitted the training data more closely while generalising worse. Ridge and Lasso controlled degree-10 complexity through coefficient shrinkage, and Lasso also removed several polynomial terms. The degree-3 Lasso model with alpha 0.001 gave the lowest testing MSE and was selected as the final model.